

# Bilinear Rank as Proof Cost: Fast Matrix-Multiplication Schemes and Field-Specific Flip Search for zkML

Greg Sidebottom

*Research note — logic repo (<https://github.com/gsidebottom/logic>), matmul track, 2026-07-11. Companion to the  $3 \times 3$  additive-complexity paper (`doc/matmul_adds_paper.md`, artifacts DOI 10.5281/zenodo.21240904), the rank-48 rigidity paper, and the flower note. The engine of §5 is `src/bin/flip23p.rs` in this repository.*

## Abstract

Zero-knowledge machine learning (zkML) — proving that a neural-network inference was computed correctly without re-executing or revealing it — is bottlenecked by the prover, whose work in circuit-based proof systems is counted in **multiplication constraints**: additions and constant-coefficient linear combinations are free. This is the exact inversion of the hardware cost model that keeps fast matrix multiplication out of GPUs, and it makes **bilinear rank** — the number of multiplications in a Strassen-like scheme — the direct cost driver for matrix products between witness values (private weights; attention). Three consequences. (1) Known schemes already pay: recursive rank-48  $4 \times 4$  blocking proves a  $768 \times 768$  witness $\times$ witness product with  $\approx 4\times$  fewer constraints than the naive circuit, penalty-free because the schemes’ additions cost nothing. (2) Coefficient obstructions vanish: schemes over  $\mathbb{Q}$  with dyadic (or any rational) coefficients are exact over a prime field, and numerical stability is not a concept. (3) Most interestingly, **rank is field-dependent** — rank 47 for  $4 \times 4$  exists modulo 2 with no characteristic-0 equivalent known — and zkML fixes a short list of concrete fields (Goldilocks, BabyBear, M31, the BN254/BLS12-381 scalar fields) over which nobody has searched. We port our rational flip-graph engine to  $\mathbb{F}_p$  (`flip23p`, Goldilocks first): over a prime field the engine is strictly stronger than its  $\mathbb{Q}$  parent — no coefficient growth, no magnitude caps, and every coincidence-solving  $\lambda$  is admissible, so the coplanarity structure that rationality left mostly unusable becomes fully spendable. A verified rank-22  $3 \times 3$  or rank-47  $4 \times 4$  over a proof field would be a shippable constraint-count reduction with no analog in the characteristic-0 literature. We state the cost model precisely, including the honest carve-outs (sumcheck/GKR proves multiply matrices in  $\sim n^2$  prover work and do not care about rank; public-weight linear layers cost no constraints), inventory which of our existing artifacts transfer, and report the port’s first measurements.

## 1 zkML in one page

A **zero-knowledge proof** (ZKP) lets a prover convince a verifier that a statement holds — here, “this output is what model  $M$  computes on input  $x$ ” — without the verifier re-executing the computation, and optionally without revealing  $x$  or  $M$ . Two families dominate. **SNARKs** (succinct non-interactive arguments of knowledge; Groth16 [6], PLONK [5]) give constant-or-polylog proof sizes over elliptic-curve-pairing fields such as the BN254 and BLS12-381 scalar fields ( $\sim 254$ -bit primes). **STARKs** [2] replace pairings with hashes, gain transparency and plausible post-quantum security, and run over small “FFT-friendly” primes chosen for fast number-theoretic transforms — Goldilocks  $p = 2^{64} - 2^{32} + 1$  (Plonky2 [12]), BabyBear  $p = 2^{31} - 2^{27} + 1$  (RISC Zero [13]), and M31  $= 2^{31} - 1$  (Circle STARKs [8]). (Appendix B explains the FFT/NTT layer these fields are engineered for.)

The asymmetry that defines the field: verification is cheap, **proving is  $10^3$  to  $10^6\times$  the native computation**, all of it exact arithmetic in the proof field. zkML applications — proving a proprietary model was faithfully executed (model privacy), proving an inference inside a blockchain rollout, auditable AI where the checker is cheap — inherit that overhead on every multiply of every layer. Making matrix multiplication cheap *inside a proof* is therefore a different optimization problem from making it fast on silicon, with a different — in fact opposite — cost model.

## 2 What proving charges

The standard arithmetization, **R1CS** (rank-1 constraint systems, as in Groth16 [6]), expresses a computation as constraints of the form

$$\left(\sum_i a_i w_i\right) \times \left(\sum_i b_i w_i\right) = \left(\sum_i c_i w_i\right)$$

over the proof field, where  $w$  is the witness vector and the  $a_i, b_i, c_i$  are *constants*. Each constraint carries exactly one multiplication of witness values; the linear combinations inside the parentheses — additions, subtractions, scalings by arbitrary field constants — are free: they are folded into the constraint’s constant vectors and cost the prover nothing. PLONK-family systems [5] charge per gate/row with the same shape (custom gates and lookup arguments shift constants around but preserve the headline: **bilinear multiplications of witnesses are the metered resource**).

For an  $n \times n$  product  $C = A \cdot B$  where both operands are witnesses, each entry  $C_{ij} = \sum_k A_{ik} B_{kj}$  is a sum of  $n$  witness×witness products; a sum of  $n$  products is not one rank-1 quadratic form — it takes  $n$  constraints. The naive circuit therefore costs  $n^3$  constraints. Two situations make both operands witnesses in zkML: **private weights** (the commercially central case — the model owner proves inference without revealing the model) and **attention** ( $QK^T$  and scores  $\cdot V$  multiply two activation matrices — witness×witness even when all weights are public). Conversely, a linear layer with *public* weights is a constant linear map and costs zero multiplication constraints; rank optimization has nothing to offer there.

**A toy example,  $2 \times 2$  over  $\mathbb{F}_{17}$ .** Take witnesses  $A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ ,  $B = \begin{pmatrix} 5 & 6 \\ 7 & 0 \end{pmatrix}$ ; then  $C = A \cdot B \equiv \begin{pmatrix} 2 & 6 \\ 9 & 1 \end{pmatrix} \pmod{17}$ . The naive circuit spends one constraint per product — eight constraints of the shape  $u_1 = A_{11} \times B_{11}$  — and output sums like  $C_{11} = u_1 + u_2$  cost nothing further (they are linear). Strassen’s rank-7 scheme spends **seven**: its first product is the single constraint

$$(w_{A_{11}} + w_{A_{22}}) \times (w_{B_{11}} + w_{B_{22}}) = w_{P_1},$$

where all four operand additions sit *inside* the constraint’s linear forms, free. There is exactly **one witness per proof**: a single vector  $w$  over  $\mathbb{F}_{17}$  holding every wire value — a conventional leading 1 (which lets constraints use constants), the inputs, the seven products, the outputs. Concretely, all 20 coordinates:

$$w = (1 \mid \underbrace{1, 2, 3, 4}_A \mid \underbrace{5, 6, 7, 0}_B \mid \underbrace{8, 1, 6, 8, 0, 5, 3}_{P_1..P_7} \mid \underbrace{2, 6, 9, 1}_C).$$

The subscripted symbols above are coordinates of this one vector, and each constraint is checked by reading them off: for  $P_1$ ,  $(1 + 4) \cdot (5 + 0) = 25 \equiv 8 \checkmark$ , and likewise  $P_2 = 7 \cdot 5 = 1$ ,  $P_3 = 1 \cdot 6 = 6$ ,  $P_4 = 4 \cdot 2 = 8$ ,  $P_5 = 3 \cdot 0 = 0$ ,  $P_6 = 2 \cdot 11 = 5$ ,  $P_7 = (-2) \cdot 7 = 3$  (all mod 17). The free output combinations reproduce  $C$  exactly:  $C_{11} = P_1 + P_4 - P_5 + P_7 = 19 \equiv 2$ ,  $C_{12} = P_3 + P_5 = 6$ ,  $C_{21} = P_2 + P_4 = 9$ ,  $C_{22} = P_1 - P_2 + P_3 + P_6 = 18 \equiv 1$ . (The prover commits to  $w$  and proves the constraints hold; the verifier never sees  $w$  — that is the “zero-knowledge”. Note the naive circuit’s witness carries eight product coordinates to Strassen’s seven, so lower rank also shortens the committed vector.) Seven constraints instead of eight — 12.5% at one level, compounding to  $n^{2.807}$  under recursion — and dyadic coefficients would be equally at home:  $\frac{1}{2}$  is just the constant 9 in  $\mathbb{F}_{17}$  ( $2 \cdot 9 = 18 \equiv 1$ ). This is §3’s story in miniature; the rank-23 and rank-48 schemes are the same picture with better ratios. (Appendix A runs a complete Setup  $\rightarrow$  Prove  $\rightarrow$  Verify on a two-constraint slice of this example — the whole SNARK pipeline at hand-checkable scale.)

## 3 Bilinear rank is constraint count

A bilinear scheme of rank  $r$  computes an  $m \times m$  block product with  $r$  products of linear forms. In R1CS each product is exactly one constraint — the linear forms slide into the constraint’s free linear parts — so an  $m \times m$  witness×witness block product costs  $r$  constraints instead of  $m^3$ , and recursion compounds it:  $T(n) = r T(n/m)$ , i.e.  $n^{\log_m r}$  constraints total, **with zero cost for the combination additions at every level**. The additions that price fast schemes out of silicon are literally free here; there is no numerical-stability tax because arithmetic is exact; and coefficients may be any field constants — dyadic  $\frac{1}{2}$ ’s are simply the constant  $(p + 1)/2 \pmod{p}$ .

Idealized constraint counts for a witness×witness product (recursion to scalar base; real circuits stop at a base tile, which shifts absolute numbers but not the ordering):

| scheme (exponent)                          | $n = 768$          | vs naive    | $n = 4096$            | vs naive    |
|--------------------------------------------|--------------------|-------------|-----------------------|-------------|
| naive (3.000)                              | $4.53 \times 10^8$ | 1.0×        | $6.87 \times 10^{10}$ | 1.0×        |
| $3 \times 3 : 23$ (2.854)                  | $1.72 \times 10^8$ | 2.6×        | $2.05 \times 10^{10}$ | 3.4×        |
| Strassen $2 \times 2 : 7$ (2.807)          | $1.26 \times 10^8$ | 3.6×        | $1.39 \times 10^{10}$ | 4.9×        |
| $4 \times 4 : 48$ ( <b>2.792</b> )         | $1.14 \times 10^8$ | <b>4.0×</b> | $1.23 \times 10^{10}$ | <b>5.6×</b> |
| $3 \times 3 : 22$ (2.814), <i>if found</i> | $1.31 \times 10^8$ | 3.4×        | $1.46 \times 10^{10}$ | 4.7×        |
| $4 \times 4 : 47$ (2.777), <i>if found</i> | $1.03 \times 10^8$ | 4.4×        | $1.08 \times 10^{10}$ | <b>6.4×</b> |

**A commutative carve-in.** One refinement the table does not yet fold in: bilinear rank assumes non-commuting entries (that is what makes block recursion valid), but **R1CS witnesses are field elements and commute** — so at the *base tile* of a recursion the true cost is the smaller commutative multiplicative complexity. Rosowski [22] multiplies  $3 \times 3$  in **21** products over a commutative ring (vs bilinear 23; Makarov’s earlier 22), and in general odd- $n$  tiles cost  $n(n^2 + 2n - 1)/2$ :  $5 \times 5$  in 85 (vs 93),  $7 \times 7$  in 217 (vs 250). Blocks do not commute, so interior levels must stay bilinear — a base-tile lever only (and  $2 \times 2$  gains nothing: 7 is optimal even commutatively) — but any recursion bottoming on  $3 \times 3$  tiles saves  $21/23 \approx 8.7\%$  of its *entire* constraint count; the optimal tile choice is a small dynamic program we leave as a refinement. Flip-graph search has recently been extended to commutative schemes [23], so §5’s discovery machinery has a commutative analogue if the base-tile hunt warrants it. Quantified against Appendix B.5’s bilinear wins: stopping the rank-48 recursion at an odd tile  $t$  and paying Rosowski’s  $R(t)$  multiplies the total by  $R(t)/t^{2.792} - 0.977$  at  $t=3$ , 0.950 at  $t=5$ , **0.947** at  $t=7$  (the optimum) — so today’s commutative ceiling is a  $\approx 5.3\%$  saving, smaller than a rank-47’s 13.5% or a rank-21’s 19%, for the structural reason one should expect: a bilinear record improves the *exponent* and compounds through every level, while commutativity improves one *constant* at the bottom. It stacks multiplicatively with any bilinear win, costs no new mathematics, and — the pricing surprise — is extremely sensitive at the smallest tile: each  $-1$  on the  $3 \times 3$  commutative count is worth 4.65% of the entire pipeline ( $3^{2.792}$  is small), so a  $3 \times 3$  commutative scheme with 20 products would deliver 7.0% and with 19 products 11.6% — rank-47-class wins from a search space minuscule beside the bilinear ones, with known lower bounds leaving room in the mid-teens. We know of no deployed ZK system exploiting fast multiplication at all — circuit-based practice proves matmul naively, and the systems that escape  $n^3$  do so by *verification* (Freivalds-style random projection, the sumcheck line of §4), not by faster multiplication — so both levers, the exponent and the tile, are currently unshipped.

Two readings. First, **the wins in the top half are available today**: the rank-48 scheme [3, 10] with dyadic coefficients drops into any R1CS/PLONK matmul gadget as-is (our repository carries the verified scheme and its minimized linear networks). Second, the *marginal* rows show why new records matter here more than in silicon: each rank step is a few percent per recursion level, compounding — and unlike hardware, nothing eats the margin.

## 4 The honest carve-outs

Rank is **not** the lever everywhere:

- **Sumcheck/GKR provers do matmul in  $\sim n^2$  prover field-ops** (Thaler [4]; GKR [7]); zkCNN [9] and successors build dedicated zkML provers on this line, and where such a prover applies, bilinear rank is irrelevant. The rank lever applies to the (large) circuit-based world: general-purpose SNARK toolchains, zkVMs executing matmul as arithmetic, PLONK/R1CS pipelines such as EZKL [11], and folding-based IVC (Nova [14]) whose per-step cost is the step circuit’s constraint count.
- **Public-weight linear layers are free**; rank helps witness×witness products only — private weights and attention.
- **Constraint count is not wall-clock**: prover time also includes commitments (multi-scalar multiplications / hashing); fewer constraints shrink those too, but constants differ by system.

- **Nonlinearities dominate some workloads** (range checks, lookups for ReLU/quantization); the matmul share is large in attention/convolution-heavy models, smaller in lookup-heavy quantized pipelines.

## 5 Field-specific rank: the open frontier, and flip23p

Bilinear rank depends on the coefficient field. The emblem: AlphaTensor’s rank-47  $4 \times 4$  scheme exists over  $\mathbb{F}_2$  [3], no characteristic-0 rank-47 is known, and our companion work proved the only  $\mathbb{Q}$ -usable rank-48 class *rigid* under a large certified move system — evidence that characteristic 0 is genuinely constrained in ways a fixed finite field need not be. zkML nominates a short, concrete list of fields that matter commercially — Goldilocks, BabyBear, M31, BN254-Fr, BLS12-381-Fr — and, to our knowledge, **no rank search has ever been run over any of them**: the flip-graph literature works over  $\mathbb{F}_2$  and small extensions [15], the explicit records over  $\mathbb{Z}/\mathbb{Q}$ . A verified  $3 \times 3$  rank-22 — or  $4 \times 4$  rank-47 — over Goldilocks would be a new kind of record with an immediate consumer: 2 to 4% fewer constraints per recursion level in deployed proof systems, compounding as in §3.

**flip23p** is our rational flip engine (splits,  $\lambda$ -flips, reductions, canonical gauge, exact Brent verification — the machinery of the companion papers) re-based onto  $\mathbb{F}_p$ , Goldilocks first. Over a prime field the engine is *strictly stronger* than its  $\mathbb{Q}$  parent:

1. **No coefficient growth.**  $\mathbb{Q}$ -walks need magnitude caps and dyadic bookkeeping; field elements don’t grow. Every cap in the  $\mathbb{Q}$  engine is deleted, not loosened.
2. **Every solved flip is admissible.** Over  $\mathbb{Q}$ , a coincidence ( $f_i + \lambda f_j \propto f_m$ , the Cramer-solved move that makes flips productive) is usable only when  $\lambda$  is dyadic — the filter discarded most solutions. Over  $\mathbb{F}_p$  every nondegenerate solution is a legal move: the coplanarity structure (“H4’s raw material” in the flower note) becomes fully spendable.
3. **The gauge simplifies:** monic normalization (leading coefficient 1, scalars folded into the  $c$ -slot); proportional  $\Leftrightarrow$  equal, with none of  $\mathbb{Q}$ ’s content/sign bookkeeping.

First measurements (2026-07-11; §8): the 55-addition record scheme loads and verifies over Goldilocks (729 Brent equations mod  $p$ ); the census gate matches the  $\mathbb{Q}$  engine exactly (9 shared pairs, weight 177, distinct-rows 40). One honest surprise: the seed’s solved-move silence is *not* the dyadic filter’s fault — even with all field  $\lambda$  available, its 9 shared pairs admit no coincidence solution; the coplanarity geometry genuinely fails there. Mobility arrives at once elsewhere: a 25-second smoke storm from the most mobile census seed executed 1.3M moves with 369K solved flips and  $\sim 10$ K reductions — a far higher solved-flip fraction than the  $\mathbb{Q}$  storm ever achieved, the freed  $\lambda$  paying immediately. No sub-23 rank has appeared in smoke tests (nor was one expected at that effort); the campaigns are the point.

## 6 What transfers from the existing program

- **Engines and discipline.** flip48/flip23’s storm, closure, census, novelty and thin-storm machinery ports mechanically (v1 of **flip23p** carries storm/census/closures; the beam-chase and exhaustive campaign modes follow). The operational rules hard-won in the companion papers — completion logs, full-frontier dumps, control baselines, budget caps — apply verbatim.
- **Seeds.** Every scheme we hold is a valid  $\mathbb{F}_p$  scheme: the 17,376 database classes, our 53 lifted classes, the 46 storm-new classes, and the 467 mod-2-invisible dyadic  $\mathbb{Q}$ -schemes (denominators are field constants). The mobility census that ranked  $\mathbb{Q}$ -seeds re-runs unchanged.
- **Verified gadgets.** The Lean pipeline that certified the 55-add scheme extends naturally: a scheme-to-circuit emitter (circom / Halo2 templates for the 23- and 48-multiplication blocks) with a machine-checked correctness certificate is a deliverable the proof-systems world actually values.
- **What does not transfer:** the additive-complexity results (55 adds, the no-54 theorem). Additions are free here; that program’s value stays in classical hardware and in witness-generation speed.

## 7 Program

(1) Goldilocks campaigns: storm + closure portfolios over the census-ranked seeds; targets, in order of plausibility  $\times$  value: any new-class harvest over  $\mathbb{F}_p$ , rank 22 at  $3 \times 3$ , rank 47 at  $4 \times 4$  (the  $4 \times 4$  port is the same edit at 16 coordinates). (2) Small-field variants: BabyBear and M31 (31-bit arithmetic,  $\sim 4\times$  faster walks), then BN254-Fr. (3) Rectangular tiles: transformer shapes want  $\langle m, n, k \rangle$  records; the engine generalizes as flip48  $\rightarrow$  flip23 did. (4) Verified gadget emitter + benchmark note: constraint-count measurements in an EZKL-style stack, naive vs recursive-scheme, with Lean-certified gadget templates.

## 8 Reproduction

```
cargo build --release --bin flip23p
./target/release/flip23p --census --dir matmul/mm23      # gates
./target/release/flip23p --dir matmul/seeds23/13a5bd4n \
  --seconds 25 --threads 4 --out matmul/found23p         # smoke storm
# any verified rank <= 22 over Goldilocks lands in
# found23p/RECORDP_rank*.txt and is announced loudly
```

## Appendix A. Setup, Prove, Verify — a complete toy proof

This appendix runs the entire SNARK pipeline, Groth16-shaped, on a two-constraint slice of §2’s example, every number over  $\mathbb{F}_{17}$ . One honest simplification, flagged where it occurs: real systems wrap the values below in elliptic-curve encodings (“in the exponent”), which is what makes hiding cryptographic; the *arithmetic* —  $\text{R1CS} \rightarrow \text{QAP} \rightarrow \text{divisibility check at a secret point}$  — is shown exactly as real provers compute it.

**The statement.** For reference, the full toy matrices of §2, whose entries supply every private value below:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 0 \end{pmatrix}, \quad C = A \cdot B \equiv \begin{pmatrix} 2 & 6 \\ 9 & 1 \end{pmatrix} \pmod{17}.$$

The prover claims: “I know private values  $A_{11}, A_{22}, B_{11}, B_{12}, B_{22}$  (here 1, 4, 5, 6, 0 — the marked entries of  $A$  and  $B$ ) such that  $P_1 = (A_{11} + A_{22})(B_{11} + B_{22}) = 8$  and  $P_3 = A_{11}(B_{12} - B_{22}) = 6$ .” The claimed products (8, 6) are **public**; the five inputs stay private. The witness (public part first):

$$w = (1, P_1=8, P_3=6 \mid A_{11}=1, A_{22}=4, B_{11}=5, B_{12}=6, B_{22}=0)$$

(Why no  $A_{12}, A_{21}, B_{21}$  here, when §2’s full witness carries every entry? Because this appendix proves only the two-constraint slice: a witness coordinate earns its place by being *wired into some constraint*, and  $c_1, c_2$  below never touch those entries. Including them would just add all-zero rows to the QAP table — their coefficient polynomials would be identically 0 on all three sides. In the full 7-constraint Strassen proof all eight inputs appear, since e.g.  $M_5 = (A_{11} + A_{12})B_{22}$  wires in  $A_{12}$ . Note the contrast:  $B_{22} = 0$  *is* present despite its value being zero — membership is about the circuit’s wiring, not the value.)

The two constraints (check:  $5 \cdot 5 = 25 \equiv 8$ ,  $1 \cdot 6 = 6$ ):

$$\begin{aligned} c_1 &: (w_{A_{11}} + w_{A_{22}}) \times (w_{B_{11}} + w_{B_{22}}) = w_{P_1} \\ c_2 &: (w_{A_{11}}) \times (w_{B_{12}} - w_{B_{22}}) = w_{P_3} \end{aligned}$$

**Step 0 — constraints become one polynomial identity (the QAP).** QAP stands for **Quadratic Arithmetic Program** [19] — the standard compilation of an R1CS into a single polynomial divisibility test, so that “all constraints hold” can later be checked at *one point* instead of once per constraint. Three sub-steps.

(a) *Read off each constraint’s coefficients.* A witness **coordinate** is one entry of the vector  $w$  (such as  $w_{A_{11}}$ ); each constraint assigns every coordinate three constant **coefficients** — its multiplier in the left factor, the right factor, and the output side. Written out in full:

$$\begin{aligned} c_1 &: \text{left} = 1 w_{A_{11}} + 1 w_{A_{22}} \quad (\text{every unlisted coordinate: } 0) \\ &\quad \text{right} = 1 w_{B_{11}} + 1 w_{B_{22}}, \quad \text{out} = 1 w_{P_1} \\ c_2 &: \text{left} = 1 w_{A_{11}}, \quad \text{right} = 1 w_{B_{12}} + (-1) w_{B_{22}}, \quad \text{out} = 1 w_{P_3} \end{aligned}$$

(b) *Build the Lagrange interpolation basis.* Assign constraint  $c_j$  to the evaluation point  $x = j$ . The **Lagrange basis polynomial**  $L_j$  is the unique lowest-degree polynomial equal to 1 at its own point and 0 at every other; the general formula  $L_j(x) = \prod_{k \neq j} \frac{x-k}{j-k}$  gives, for our points  $\{1, 2\}$ ,

$$L_1(x) = \frac{x-2}{1-2} = 2-x, \quad L_2(x) = \frac{x-1}{2-1} = x-1,$$

with  $L_1(1) = 1, L_1(2) = 0$  and  $L_2(1) = 0, L_2(2) = 1$  (over  $\mathbb{F}_{17}$  the division by  $1-2 = -1$  is multiplication by 16 — every nonzero denominator is invertible in a field). These are building blocks: the polynomial taking values  $(v_1, v_2)$  at  $(1, 2)$  is exactly  $v_1 L_1 + v_2 L_2$ .

(c) *Interpolate each coordinate's coefficients.* For coordinate  $k$ , collect its left-factor coefficients across constraints as the value list (value in  $c_1$ , value in  $c_2$ ) and set  $A_k(x) = (\text{coeff in } c_1) L_1(x) + (\text{coeff in } c_2) L_2(x)$ ; likewise  $B_k$  from right-factor and  $C_k$  from output coefficients. Two worked rows:  $A_{22}$  has left-coefficients  $(1, 0)$ , so its A-poly is  $1 \cdot L_1 + 0 \cdot L_2 = 2 - x$ ;  $B_{22}$  has right-coefficients  $(1, -1)$ , so its B-poly is  $L_1 - L_2 = (2 - x) - (x - 1) = 3 - 2x$ . The full table:

| coord    | A-side of  | A-poly  | B-side of        | B-poly   | C-side of | C-poly  |
|----------|------------|---------|------------------|----------|-----------|---------|
| $A_{11}$ | $c_1, c_2$ | 1       | —                | 0        | —         | 0       |
| $A_{22}$ | $c_1$      | $2 - x$ | —                | 0        | —         | 0       |
| $B_{11}$ | —          | 0       | $c_1$            | $2 - x$  | —         | 0       |
| $B_{12}$ | —          | 0       | $c_2$            | $x - 1$  | —         | 0       |
| $B_{22}$ | —          | 0       | $c_1(+), c_2(-)$ | $3 - 2x$ | —         | 0       |
| $P_1$    | —          | 0       | —                | 0        | $c_1$     | $2 - x$ |
| $P_3$    | —          | 0       | —                | 0        | $c_2$     | $x - 1$ |

Weight each polynomial by its witness value and sum:

$$A(x) = 1 \cdot 1 + 4(2 - x) = 9 - 4x, \quad B(x) = 5(2 - x) + 6(x - 1) = x + 4, \quad C(x) = 8(2 - x) + 6(x - 1) = 10 - 2x.$$

Sanity: at  $x=1$ :  $A \cdot B = 25 \equiv 8 = C$  ( $c_1$  ✓); at  $x=2$ :  $A \cdot B = 6 = C$  ( $c_2$  ✓). **Both constraints hold**  $\iff A(x)B(x) - C(x)$  **vanishes at**  $x = 1, 2$   $\iff$  **it is divisible by**  $Z(x) = (x - 1)(x - 2)$ . Over  $\mathbb{F}_{17}$ :

$$P(x) = AB - C = -4x^2 - 5x + 26 \equiv 13x^2 + 12x + 9, \quad Z(x) = x^2 - 3x + 2, \quad H(x) = P/Z = 13$$

(exact division, remainder 0).

Where  $Z$  comes from.  $Z$  is the **vanishing polynomial** of the constraint points. Constraint  $c_j$  holds exactly when  $P(j) = A(j)B(j) - C(j) = 0$  — that is, when  $(x - j)$  divides  $P$ . Both constraints hold exactly when  $(x - 1)$  and  $(x - 2)$  *both* divide  $P$ , i.e. when their product  $Z(x) = (x - 1)(x - 2) = x^2 - 3x + 2$  does. (With  $m$  constraints,  $Z = \prod_{j=1}^m (x - j)$ : one root per constraint — “ $P$  vanishes wherever a constraint lives.”)

How  $H = 13$  was computed. Ordinary polynomial division over  $\mathbb{F}_{17}$ , which here collapses to one step:  $P$  and  $Z$  both have degree 2, so  $H$  is the constant (leading coefficient of  $P$ )  $\div$  (leading coefficient of  $Z$ )  $= 13 \div 1 = 13$ , and exactness is one multiplication:

$$13 Z = 13x^2 + 13(-3)x + 13 \cdot 2 = 13x^2 - 39x + 26 \equiv 13x^2 + 12x + 9 \pmod{17} = P \quad (\text{remainder } 0). \quad \checkmark$$

The exactness is not an algebraic accident; it is the entire content of the proof: **division comes out clean precisely because the witness satisfies both constraints**. Tamper with the claim — say the prover asserts  $P_1 = 9$  instead of 8 — and  $C(x)$  becomes  $9(2 - x) + 6(x - 1)$ , so  $P(1) = 25 - 9 = 16 \neq 0$ : now  $(x - 1)$  no longer divides  $P$ , no valid  $H$  exists at all, and the cheater's only remaining hope is faking the divisibility check at the one hidden point  $\tau$  — exactly what Step 3's Schwartz-Zippel argument makes overwhelmingly unlikely.

The secret knowledge is compressed into: “I can exhibit witness-weighted  $A, B, C$  and a quotient  $H$  with  $AB - C = HZ$ .”

**Step 1 — Setup (the trusted ceremony).** Sample a secret point, say  $\tau = 5$  — the “toxic waste” — and publish the powers of  $\tau$  and the QAP polynomials evaluated at  $\tau$ , each wrapped in an encoding  $\text{enc}(\cdot)$  that supports additions and scalings but hides the value — defined concretely just below. Then **destroy**  $\tau$ : anyone holding it can forge proofs — hence the multi-party ceremonies around Groth16, and the transparent (no- $\tau$ ) alternatives like STARKs. For our toy, the published material (the **common reference string**, CRS) is *witness-independent* — one ceremony serves every future proof of this circuit — and consists of the encoded evaluations at  $\tau = 5$  of each *coordinate* polynomial from Step 0’s table, plus  $Z(5)$  and encodings of the powers of  $\tau$  that  $H$  may need (here just  $\tau^0$ , since our  $H$  has degree 0). Spelled out:

$$\begin{array}{lll} A_{A_{11}}(5) = 1 & A_{A_{22}}(5) = 2 - 5 \equiv 14 & \\ B_{B_{11}}(5) \equiv 14 & B_{B_{12}}(5) = 4 & B_{B_{22}}(5) = 3 - 10 \equiv 10 \\ C_{P_1}(5) \equiv 14 & C_{P_3}(5) = 4 & Z(5) = 4 \cdot 3 = 12 \end{array}$$

(every unlisted coordinate’s polynomial is identically 0). Note what is **not** published: anything witness-dependent —  $A(x)$ ,  $B(x)$ ,  $C(x)$ ,  $P(x)$ ,  $H(x)$  belong to the prover, not the setup.

**What  $\text{enc}(\cdot)$  is.** Exponentiation in a group where the reverse direction — recovering the exponent, the *discrete logarithm* — is computationally infeasible: fix a public generator  $g$  and set  $\text{enc}(v) = g^v$ . Real systems use elliptic-curve groups of size  $\sim 2^{256}$ ; the toy admits an honest miniature. Inside the integers mod 103 (a prime), the powers of  $g = 72$  form a subgroup of order exactly 17 —  $72^{17} \equiv 1 \pmod{103}$ , no smaller power is 1 — so exponents live precisely in  $\mathbb{F}_{17}$ :

$$\text{enc}(v) = 72^v \pmod{103} : \quad \begin{array}{l} \text{enc}(0) = 1, \text{ enc}(1) = 72, \text{ enc}(4) = 23, \text{ enc}(6) = 61, \text{ enc}(9) = 81, \\ \text{enc}(10) = 64, \text{ enc}(12) = 13, \text{ enc}(13) = 9, \text{ enc}(14) = 30. \end{array}$$

What Setup *literally* publishes is this column:  $\text{enc}(1) = 72$ ,  $\text{enc}(14) = 30$ ,  $\text{enc}(4) = 23$ ,  $\text{enc}(10) = 64$  for the coordinate polynomials,  $\text{enc}(12) = 13$  for  $Z(5)$  — the raw values displayed above were narrator’s courtesy. (That  $\text{enc}(12) = 13$  numerically equals  $H$ ’s value is pure coincidence.) Two properties do all the work: (i) **hiding** — recovering  $v$  from  $72^v \pmod{103}$  is the discrete-log problem, brute-forceable at order 17 (the toy’s one dishonesty: 17 candidates) but infeasible at order  $\sim 2^{256}$ , which is why publishing encodings of powers of  $\tau$  does not reveal  $\tau$ ; (ii) **linearity passes through** —  $\text{enc}(a)\text{enc}(b) = g^{a+b} = \text{enc}(a+b)$  and  $\text{enc}(v)^c = \text{enc}(cv)$  for known  $c$ , so anyone can form known-coefficient linear combinations of hidden values without decoding. What the encoding can *never* do is multiply two hidden values; that asymmetry is the crypto-layer origin of this paper’s cost model — linear operations ride through for free, each hidden $\times$ hidden product must be paid for as a constraint, and the verifier’s one pairing (Step 3) grants exactly one such product, just enough for the final check.

**Step 2 — Prove.** The prover, holding  $w$ , computes the *encoded* evaluations — each a **linear combination** of published encodings with witness coefficients (this is why R1CS’s free-linear-combination structure is the whole design):

$$\begin{aligned} A(5) &= w_{A_{11}} A_{A_{11}}(5) + w_{A_{22}} A_{A_{22}}(5) = 1 \cdot 1 + 4 \cdot 14 = 57 \equiv 6, \\ B(5) &= 5 \cdot 14 + 6 \cdot 4 + 0 \cdot 10 = 94 \equiv 9, \quad C(5) = 8 \cdot 14 + 6 \cdot 4 = 136 \equiv 0, \quad H(5) = 13. \end{aligned}$$

Cross-check by direct evaluation (which only we, the omniscient narrators, can do):  $A(x) = 9 - 4x$  at 5 gives  $-11 \equiv 6 \checkmark$ ,  $B = x + 4$  gives  $9 \checkmark$ ,  $C = 10 - 2x$  gives  $0 \checkmark$  — the same numbers, but the prover’s route touched only published encodings and witness values, never  $\tau$  itself. In encoded form — all a real prover ever holds — the same combinations run *inside the group*, by property (ii):

$$\begin{aligned} \text{enc}(A(5)) &= \text{enc}(A_{A_{11}}(5))^{w_{A_{11}}} \text{enc}(A_{A_{22}}(5))^{w_{A_{22}}} = 72^1 \cdot 30^4 = 72 \cdot 8 \equiv 61 \pmod{103} = \text{enc}(6), \\ \text{enc}(B(5)) &= 30^5 23^6 64^0 \equiv 81 = \text{enc}(9), \quad \text{enc}(C(5)) = 30^8 23^6 \equiv 1 = \text{enc}(0), \\ \text{enc}(H(5)) &= \text{enc}(\tau^0)^{13} = 72^{13} \equiv 9 = \text{enc}(13). \end{aligned}$$

The transmitted proof is literally those four group elements,  $\pi = (\mathbf{61}, \mathbf{81}, \mathbf{1}, \mathbf{9})$ , and nothing else. (Both  $C$ -coordinates here happen to be public outputs, so the verifier could rebuild  $\text{enc}(C(5))$  itself; in general the proof carries the private part  $C_{\text{priv}}$ .) In Groth16, after optimizations, the proof compresses to three curve points,  $\sim 200$  bytes, *independent of circuit size*. The prover never learns  $\tau$ .

**Step 3 — Verify.** The verifier reconstructs the *public* part of  $C(\tau)$  itself from the claimed outputs (8,6) — the prover cannot lie about what is being claimed — then checks **one multiplicative relation**. In real systems this is one *pairing* equation: a pairing satisfies  $e(g^a, g^b) = e(g, g)^{ab}$ , multiplying two hidden values exactly once — the single multiplication the encoding ever grants. Our mod-103 toy group has no pairing, so here (the toy’s one honest deviation) we decode and check in the clear:

$$A(\tau)B(\tau) - C(\tau) \stackrel{?}{=} H(\tau)Z(\tau) : \quad 6 \cdot 9 - 0 = 54 \equiv 3, \quad 13 \cdot 12 = 156 \equiv 3. \quad \checkmark$$

**Why this convinces (soundness).** A cheating prover without a valid witness needs  $AB - C$  divisible by  $Z$ ; the best it can do is fake the relation at the single hidden point  $\tau$ . Two distinct low-degree polynomials agree at a random point with probability  $\leq \deg / |\mathbb{F}|$  — about  $2/17$  in the toy (which is why  $\mathbb{F}_{17}$  is a classroom field), about  $2^{-250}$  over a real 254-bit field. Not knowing  $\tau$ , the prover cannot aim: Schwartz–Zippel is the heart of the construction.

**Why it reveals nothing (zero-knowledge).** The verifier sees only encodings, never witness coordinates. (Full Groth16 additionally randomizes each proof with masking multiples of  $Z(x)$  so two proofs of the same witness look independent; omitted for clarity.)

**The connection to rank, one last time.** Each constraint became one interpolation point, one row of the QAP table, one share of prover work and setup size. Prove a witness  $\times$  witness matrix product with a rank- $r$  scheme and this whole pipeline is  $r$  constraints per block instead of  $m^3$  — the scheme’s additions never appear as constraints at any stage; they live inside the linear combinations, which the encodings support for free.

## Appendix B. The FFT/NTT layer — how the polynomial pipeline scales

Appendix A ran the whole pipeline at two constraints, where every polynomial step was hand-arithmetic: interpolate from two points, one product, one division. A real circuit has *millions* of constraints, and each of those steps done naively costs  $O(m^2)$  field operations. The workhorse that makes them  $O(m \log m)$  is the fast Fourier transform run over  $\mathbb{F}_p$  — the **NTT** (number-theoretic transform). This appendix shows it working at toy scale in the same  $\mathbb{F}_{17}$ , then gives the production numbers.

**B.1 Where transforms appear.** Four steps of Appendix A become transform calls at scale: (1) *interpolation* (Step 0): witness columns  $\rightarrow$  the coefficient polynomials  $A, B, C$  — one **inverse NTT** each replaces the  $O(m^2)$  Lagrange formulas; (2) *products* such as  $A(x)B(x)$ : NTT both onto a larger domain, multiply **pointwise**, inverse-NTT back; (3) *the quotient*  $H = P/Z$ : pointwise division on a coset — see B.3, where this becomes almost comically cheap; (4) (STARKs) the *low-degree extension*: re-evaluate the trace on a 2 to  $8 \times$  larger domain — Reed–Solomon encoding, again NTTs — before Merkle-hashing and FRI.

**B.2 The transform itself, at size 4 in  $\mathbb{F}_{17}$ .** The classical FFT evaluates a polynomial at the complex  $n$ -th roots of unity  $e^{2\pi i k/n}$ ; the sines and cosines are merely how  $\mathbb{C}$  *parametrizes* its roots of unity. The algorithm needs only three algebraic facts:  $\omega^n = 1$ ,  $\omega^{n/2} = -1$ , and “squaring the  $n$ -th roots gives the  $(n/2)$ -th roots” (what lets the problem halve recursively). These are radix-2 requirements, so  $n$  is always a power of two — circuits pad to the next  $2^k$  with dummy constraints rather than ever running an odd-length transform (mod 17 odd orders don’t even exist: every element order divides 16). Any field with an element of order  $n$  runs the identical recursion — exactly, with no rounding. In  $\mathbb{F}_{17}$ ,  $p - 1 = 16$ , so orders up to 16 exist;  $\omega = 4$  has order 4:

$$\omega = 4 : \quad \omega^1 = 4, \omega^2 = 16 \equiv -1, \omega^3 = 13, \omega^4 = 1; \quad D = \{1, 4, 16, 13\}.$$

Run it on Appendix A’s own  $P(x) = 13x^2 + 12x + 9$ . Split by even/odd coefficients —  $P(x) = E(x^2) + xO(x^2)$  with  $E(y) = 9 + 13y$ ,  $O(y) = 12$ . Now watch the domain collapse: the four points square pairwise onto just two values,  $1^2 = 1$ ,  $4^2 = 16$ ,  $16^2 = 256 \equiv 1$ ,  $13^2 = 169 \equiv 16$  — so  $E$  and  $O$  need evaluating only at



$\{1, 16\} = \{1, -1\}$ . That collapse *is* the halving. Combine with the butterfly  $P(x) = E(x^2) + xO(x^2)$  at each of the four  $x$ :

$$\begin{aligned} E(1) &= 22 \equiv 5, & E(16) &= 9 + 208 \equiv 13, & O &\equiv 12 \text{ everywhere;} \\ P(1) &= 5 + 1 \cdot 12 \equiv 0, & P(4) &= 13 + 4 \cdot 12 \equiv 10, & P(16) &= 5 + 16 \cdot 12 \equiv 10, & P(13) &= 13 + 13 \cdot 12 \equiv 16. \end{aligned}$$

Six multiplications instead of the naive twelve (naively each point costs three:  $x \cdot x$ ,  $13 \cdot x^2$ ,  $12 \cdot x$ ). At this size even the six flatter to deceive —  $13 \cdot 1$  and  $1 \cdot 12$  are multiplications by 1, and since  $16 \equiv -1$ ,  $13 \equiv -4$ , two more are negations or reuses — but that is exactly the right lesson: the honest claim is asymptotic. The recursion satisfies  $T(n) = 2T(n/2) + O(n) = O(n \log n)$  against  $O(n^2)$ : invisible at  $n = 4$ , a  $\sim 10^5 \times$  factor at  $n = 2^{20}$ . (Note  $P(1) \equiv 0$ :  $x=1$  is one of Appendix A’s constraint points, and the transform *displays* constraint  $c_1$  holding. Real systems put **all** constraints on such a domain, so the whole R1CS check is visible in one spectrum.) The inverse transform is the same butterfly with  $\omega^{-1} = 13$  and a global factor  $n^{-1} = 4^{-1} \equiv 13$ .

**B.3 Why roots of unity: the vanishing polynomial collapses.** Appendix A built  $Z(x) = (x-1)(x-2)$  by multiplying linear factors — fine at 2 points, hopeless at  $2^{20}$ . On the domain  $D$  above (we write  $D$  for the domain —  $H$  is already taken by the quotient polynomial; much of the SNARK literature does the opposite,  $H$  for the domain and  $h(x)$  for the quotient),

$$Z_D(x) = x^4 - 1 \quad (\text{generally } x^n - 1),$$

one subtraction to evaluate anywhere. Better still, the quotient  $H = P/Z$  is computed on a *shifted coset*  $g \cdot D$  where  $Z$  never vanishes — and there it is not merely cheap but **constant**: on  $3D = \{3, 12, 14, 5\}$  every point satisfies  $x^4 = 3^4 \equiv 13$ , so  $Z_D \equiv 12$  across the entire coset, and “divide by  $Z$ ” is one multiplication by  $12^{-1} \equiv 10$ . Appendix A’s polynomial long division becomes: NTT  $P$  onto the coset, scale by a constant, inverse NTT.

**B.4 Two-adicity — why the proof fields look the way they do.** A radix-2 NTT of size  $2^k$  needs an element of order  $2^k$ , i.e.  $2^k \mid p-1$ . That single divisibility requirement shapes the field zoo:

| field                 | $p$               | $p-1$ factors as                                            | max radix-2 NTT |
|-----------------------|-------------------|-------------------------------------------------------------|-----------------|
| toy $\mathbb{F}_{17}$ | 17                | $2^4$                                                       | 16              |
| Goldilocks            | $2^{64}-2^{32}+1$ | $2^{32}(2^{32}-1)$                                          | $2^{32}$        |
| BabyBear              | $2^{31}-2^{27}+1$ | $2^{27} \cdot 3 \cdot 5$                                    | $2^{27}$        |
| BN254-Fr              | $\sim 2^{254}$    | $2^{28} \cdot (\text{odd})$                                 | $2^{28}$        |
| M31                   | $2^{31}-1$        | $2 \cdot 3^2 \cdot 7 \cdot 11 \cdot 31 \cdot 151 \cdot 331$ | <b>2</b>        |

M31 is the cautionary tale: two-adicity 1, no radix-2 domains at all — the reason Circle STARKs [8] exist (they run the transform on the unit circle  $x^2 + y^2 = 1$  over  $\mathbb{F}_p$ , which has  $2^{31}$  points with perfect 2-adic structure). Two further design notes. *Exactness*: a floating-point FFT’s rounding is harmless in signal processing and fatal here — one wrong field element breaks the divisibility identity — while the NTT is exact by construction. *Reduction cost*: the NTT does one modular reduction per multiply, so proof fields are engineered for cheap reduction (Goldilocks reduces with shifts and adds; BabyBear/M31 fit 32-bit lanes and vectorize).

**B.5 Production scale.** Order-of-magnitude anchors (2026 practice). A single proof segment typically carries  $2^{20}$  to  $2^{24}$  constraints or trace rows ( $\approx 10^6$  to  $1.6 \times 10^7$ ); STARK blowups of 2 to  $8 \times$  put the largest NTTs at  $2^{23}$  to  $2^{27}$  points — BabyBear’s  $2^{27}$  ceiling is not an accident but the binding constraint (a  $2^{24}$ -row trace at blowup 8 uses the whole two-adic budget). A Groth16 prover runs  $\approx 7$  size- $m$  FFTs plus 4 size- $m$  multi-scalar multiplications; STARK provers are NTT + hashing dominated. Computations bigger than one segment shard into thousands of segments whose proofs are aggregated by recursion. §3’s zkML numbers land here: one  $n = 4096$  witness $\times$ witness product costs  $1.23 \times 10^{10}$  constraints under rank-48 recursion —  $\approx 10^4$  segments of  $2^{20}$  — versus  $6.87 \times 10^{10}$  naive, a  $5.6 \times$  cut in segments, NTTs, and commitments alike; per-layer transformer matrices (2048 to 8192 on a side) sit squarely in the §3 table’s range. The division of labor is exact: the NTT/commitment layer fixes the **cost per constraint**, and the bilinear rank of §3 fixes **how many constraints there are**. Lower rank does not speed up the transform; it shrinks the transform.

**What a new record would buy.** Prover time is  $\approx \text{NTT} (m \log m) + \text{commitments} (m)$ , so speedup tracks the constraint ratio (the log factor barely moves). At  $n = 4096$ , folding in §3’s exponents:

- a **rank-47**  $4 \times 4$  is a drop-in upgrade of today’s best base:  $(48/47)^6 \approx 1.13\times$  fewer matmul constraints (13.5%) — at a 90% matmul share,  $\approx 12\%$  end-to-end prover time;
- a **rank-22**  $3 \times 3$  speeds  $3 \times 3$ -blocked pipelines by **1.40** $\times$  but still trails the rank-48 base ( $0.84\times$ ) — its value is scientific (first sub-23) and field-cartographic, not operational;
- a **rank-21**  $3 \times 3$  — the Strassen-beating line — would take over as the best known base outright: **1.99** $\times$  over rank-23 pipelines and **1.19** $\times$  over rank-48, i.e.  $\approx 19\%$  fewer constraints, NTTs, and committed elements than anything available today.

End-to-end wall clock scales by the circuit’s matmul share (80 to 95% in typical zkML inference), Amdahl-style.

**Larger bases (records as of July 2026).** The  $\mathbb{F}_p$ -valid records above  $4 \times 4$  — mod-2 results do *not* transfer to odd proof primes — are:  $5 \times 5$ : **93** and  $6 \times 6$ : **153** (Moosbauer–Poole [20], explicitly “over arbitrary ground fields”);  $7 \times 7$ : **250** (Sedoglavic [21], non-commutative, so any ring);  $8 \times 8$ : **336** (= Strassen  $\otimes$  rank-48; mod 2 has  $329 = 7 \cdot 47$  — another instance of the field divergence this paper’s program probes). What beating each by 1 or 2 buys same-base pipelines at  $n = 4096$  (speedup =  $(r/(r-k))^{\log_m 4096}$ ; wall clock  $\times$  matmul share as above):

| base         | record $r$ | exponent | −1    | −2     | overtakes rank-48 at |
|--------------|------------|----------|-------|--------|----------------------|
| $5 \times 5$ | 93         | 2.816    | +5.7% | +11.9% | $\leq 89$            |
| $6 \times 6$ | 153        | 2.808    | +3.1% | +6.3%  | $\leq 148$           |
| $7 \times 7$ | 250        | 2.837    | +1.7% | +3.5%  | $\leq 229$           |
| $8 \times 8$ | 336        | 2.797    | +1.2% | +2.4%  | $\leq 332$           |

Two readings. A  $-1$  is worth more at smaller bases: the recursion depth  $\log_m n$  shrinks as  $m$  grows, so a  $5 \times 5$  improvement compounds through 5.2 levels at  $n = 4096$  while an  $8 \times 8$  one gets only 4. And no  $-1$  or  $-2$  at these sizes overtakes the rank-48  $4 \times 4$  base — the nearest paths to a new best base run through  $5 \times 5 \leq 89$  ( $-4$ ) and  $8 \times 8 \leq 332$  ( $-4$ ; especially interesting because today’s 336 is *inherited* from rank-48 via Strassen, so any direct  $8 \times 8$  construction at  $\leq 332$  would beat every  $4 \times 4$ -blocked pipeline at once).

## Acknowledgments

This work was carried out in an extended interactive collaboration with Claude (Anthropic; the Fable 5 and Opus 4.8 models), which implemented the engines and drafted this text under the author’s direction and review.

## References

- [1] V. Strassen. *Gaussian elimination is not optimal*. Numer. Math. 13:354 to 356, 1969.
- [2] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev. *Scalable, transparent, and post-quantum secure computational integrity*. ePrint 2018/046.
- [3] A. Fawzi et al. *Discovering faster matrix multiplication algorithms with reinforcement learning*. Nature 610:47 to 53, 2022.
- [4] J. Thaler. *Time-optimal interactive proofs for circuit evaluation*. CRYPTO 2013.
- [5] A. Gabizon, Z. Williamson, O. Ciobotaru. *PLONK*. ePrint 2019/953.
- [6] J. Groth. *On the size of pairing-based non-interactive arguments*. EUROCRYPT 2016.
- [7] S. Goldwasser, Y. T. Kalai, G. N. Rothblum. *Delegating computation: interactive proofs for muggles*. STOC 2008.

- [8] U. Haböck, D. Levit, S. Papini. *Circle STARKs*. ePrint 2024/278.
- [9] T. Liu, X. Xie, Y. Zhang. *zkCNN*. CCS 2021.
- [10] J.-G. Dumas, C. Pernet, A. Sedoglavic. arXiv:2506.13242; A. Novikov et al. *AlphaEvolve*. arXiv:2506.13131.
- [11] EZKL. [github.com/zkonduit/ezkl](https://github.com/zkonduit/ezkl).
- [12] Polygon Zero. *Plonky2*. Whitepaper, 2022.
- [13] RISC Zero. *zkVM documentation*, 2023.
- [14] A. Kothapalli, S. Setty, I. Tzialla. *Nova*. CRYPTO 2022.
- [15] M. Kauers, J. Moosbauer. *Flip graphs for matrix multiplication*. arXiv:2212.01175.
- [16] G. Grassi et al. *Poseidon*. USENIX Security 2021.
- [17] G. Sidebottom. *A 55-addition rank-23 scheme for  $3 \times 3$  matrix multiplication*. This repository, 2026. DOI 10.5281/zenodo.21240904.
- [18] G. Sidebottom. *Rigidity of the rank-48  $4 \times 4$  scheme and The Flower*. Companion notes, this repository, 2026.
- [19] R. Gennaro, C. Gentry, B. Parno, M. Raykova. *Quadratic span programs and succinct NIZKs without PCPs*. EUROCRYPT 2013. (The QAP compilation.)
- [20] J. Moosbauer, M. Poole. *Flip graphs with symmetry and new matrix multiplication schemes*. ISSAC 2025; arXiv:2502.04514. ( $5 \times 5:93$  and  $6 \times 6:153$ , over arbitrary ground fields.)
- [21] A. Sedoglavic. *A non-commutative algorithm for multiplying  $7 \times 7$  matrices using 250 multiplications*. Preprint, 2017.
- [22] A. Rosowski. *Fast commutative matrix algorithms*. 2019, arXiv:1904.07683. ( $3 \times 3$  in 21 commutative products; odd- $n$  tiles in  $n(n^2 + 2n - 1)/2$ .)
- [23] *Exploring commutative matrix multiplication schemes via flip graphs*. 2025, arXiv:2506.22113.